

ENTREGA PROYECTO FINAL

Giovani Esteban Romero

UNIVERSIDAD DE LA SABANA MAESTRIA EN ARQUITECTURA DE SOFTWARE

Tabla de Contenido

No.	Sección	Página
1	Executive Summary	3
2	Contexto del proyecto Ecommify	3
3	Arquitectura híbrida implementada	4
4	Principales logros y métricas de éxito	5
5	Recomendaciones estratégicas clave	6
6	Problema de negocio	7
7	Objetivos y alcance	7
8	Metodología aplicada	8
9	Diagrama de arquitectura híbrida completa	8
10	Matriz de decisiones: datos en PostgreSQL vs MongoDB	8
11	Análisis del Teorema CAP aplicado por módulo	11
12	Modelos ER y de documentos	12
12.1	Diagrama ER en PostgreSQL	12
12.2	Diagrama ER en MongoDB	14
13	Resumen de implementación PostgreSQL	14
14	Resumen de implementación MongoDB	15
15	Metodología de pruebas aplicada	16
16	Resultados de pruebas de carga y escalabilidad	18
17	Análisis comparativo PostgreSQL vs MongoDB	18
18	Gráficas y métricas clave	20
19	Trade-offs del Teorema CAP y escenarios de falla	20
20	Lecciones aprendidas sobre arquitecturas híbridas	23
21	Plan de escalamiento para crecimiento 10x	24
22	Migración de free tier a producción	26
23	Tecnologías complementarias recomendadas	27
23.1	Caching	27
23.2	Search	27
23.3	Observabilidad	27
24	Estrategia de CI/CD para cambios de esquema	28
25	Cumplimiento de objetivos	29
26	Principales aprendizajes del equipo	29
27	Reflexión final sobre el proceso	29
28	Referencias	30

1. Executive Summary

El proyecto DISEÑO DE BASE DE DATOS consolidó una arquitectura híbrida para Ecommify, combinando PostgreSQL/Supabase como motor relacional para el núcleo transaccional y MongoDB Atlas como motor documental para escenarios flexibles y de alta disponibilidad. En PostgreSQL se estructuró el modelo relacional, se cargaron los datasets principales, se identificaron consultas críticas, se ejecutaron análisis de rendimiento con EXPLAIN (ANALYZE, BUFFERS) y se aplicaron optimizaciones mediante índices especializados y particionamiento mensual sobre la tabla orders, incluyendo partición DEFAULT y estrategia de migración de datos existentes.

Los resultados cuantitativos evidenciaron una mejora significativa en el desempeño del modelo relacional, reduciendo el tiempo total de ejecución de 2.786,066 ms a 340,828 ms, equivalente a una mejora aproximada del 87,77%, y disminuyendo los bloques leídos de 1.103 a 123, con una reducción cercana al 88,85%. Estas mejoras reflejan una menor dependencia de escaneos secuenciales, mejor aprovechamiento de índices, reducción del volumen de datos procesado y mayor eficiencia en operaciones.

En MongoDB, con base en el análisis de índice de concentración, se definió una estrategia orientada a documentos, disponibilidad y eficiencia de lectura, considerando un réplica set multi-AZ de tres nodos, políticas diferenciadas de Read/Write Preference, consistencia eventual, replication lag y mitigaciones como lectura desde primary y causal consistency sessions para operaciones críticas.

Esta arquitectura propuesta permite aprovechar las fortalezas de cada tecnología: PostgreSQL para integridad, consistencia transaccional y análisis relacional; MongoDB para flexibilidad documental, consultas agregadas y resiliencia. En conjunto, la solución mejora el rendimiento, la escalabilidad, la disponibilidad y la gobernabilidad del modelo de datos, manteniendo una aproximación viable dentro de las restricciones del free tier mediante carga controlada, índices selectivos, monitoreo y separación clara de responsabilidades entre motores.

1. Contexto del proyecto Ecommify

Ecommify corresponde a un caso de estudio de comercio electrónico orientado al diseño, implementación y optimización de una arquitectura de base de datos híbrida, utilizando datasets transaccionales asociados a clientes, pedidos, productos, vendedores, pagos, reseñas e información geográfica. El objetivo principal del proyecto es estructurar un modelo de datos que soporte operaciones comerciales, analíticas y logísticas, garantizando integridad, rendimiento, escalabilidad y capacidad de análisis sobre altos volúmenes de información.

La solución se abordó mediante dos motores de base de datos complementarios. PostgreSQL/Supabase se utiliza como base relacional principal para gestionar el núcleo transaccional del negocio, incluyendo pedidos, clientes, pagos, productos y relaciones entre entidades. Sobre este motor se aplican estrategias de normalización, integridad referencial, consultas críticas, análisis con EXPLAIN (ANALYZE, BUFFERS), indexación especializada y particionamiento mensual de la tabla orders. Por su parte, MongoDB Atlas se incorpora como base documental para escenarios donde se requiere mayor

flexibilidad, almacenamiento de documentos enriquecidos, consultas agregadas y análisis orientados a lectura, como reseñas, catálogos extendidos, perfiles consolidados o vistas documentales.

El proyecto también contempla la evaluación de rendimiento y disponibilidad. En PostgreSQL se identifican cuellos de botella, operaciones costosas, escaneos secuenciales y oportunidades de mejora mediante índices y particionamiento. En MongoDB se analizan aspectos de distribución, concentración de datos, réplica set multi-AZ, preferencias de lectura/escritura, consistencia eventual y estrategias de mitigación. En conjunto, Ecommify permite demostrar cómo una arquitectura híbrida puede aprovechar las fortalezas de cada motor de base de datos.

2. Arquitectura híbrida implementada.

La arquitectura híbrida implementada para Ecommify integra PostgreSQL/Supabase y MongoDB Atlas bajo un modelo de asignaciones diferenciadas, donde cada motor de base de datos se utiliza según su fortaleza técnica y el tipo de carga que debe soportar. PostgreSQL se definió como el componente principal para el núcleo transaccional, gestionando entidades estructuradas como clientes, pedidos, productos, vendedores, pagos, ítems de pedido y reseñas, garantizando integridad referencial, consistencia, trazabilidad y soporte para consultas relacionales complejas. Sobre este motor se aplicaron estrategias de optimización como índices especializados, análisis con EXPLAIN (ANALYZE, BUFFERS) y particionamiento mensual de la tabla orders, con el objetivo de mejorar el rendimiento, reducir bloques leídos y facilitar la administración histórica de datos.

MongoDB Atlas se incorporó como componente documental complementario para escenarios donde se requiere mayor flexibilidad, consultas orientadas a documentos, agregaciones y modelos de lectura optimizados. Este motor se orienta a almacenar información semiestructurada o enriquecida, como catálogos extendidos, reseñas, perfiles consolidados, documentos agregados y vistas preparadas para consulta. Adicionalmente, se consideró una configuración de réplica set multi-AZ con tres nodos, estrategias diferenciadas de lectura y escritura, control de consistencia eventual y mitigaciones como lectura desde nodo primario o sesiones con consistencia causal para operaciones sensibles.

En conjunto, la arquitectura híbrida permite separar adecuadamente las cargas transaccionales, analíticas y documentales. PostgreSQL aporta robustez, consistencia e integridad para la operación central del negocio, mientras que MongoDB aporta flexibilidad, disponibilidad y eficiencia para consultas documentales y agregadas. Esta separación mejora la escalabilidad general de la solución, reduce la presión sobre un único motor de base de datos y permite evolucionar el modelo de datos de acuerdo con los patrones de uso, volumen de información y necesidades de rendimiento de Ecommify.

Componente	Rol dentro de la arquitectura
PostgreSQL / Supabase	Núcleo transaccional, integridad relacional, consultas críticas, particionamiento e indexación

MongoDB Atlas	Modelo documental, datos semiestructurados, agregaciones, vistas de lectura y alta disponibilidad
Particionamiento	Organización mensual de pedidos para mejorar rendimiento y mantenimiento histórico
Índices	Optimización de búsquedas, filtros, joins, agregaciones y consultas textuales
Réplica set MongoDB	Disponibilidad, resiliencia y balance entre consistencia y lectura distribuida

3. Principales logros y métricas de éxito.

Se logró consolidar una arquitectura de datos basada en PostgreSQL/Supabase y MongoDB Atlas, separando las cargas transaccionales, analíticas y documentales.

En PostgreSQL se implementó un modelo relacional optimizado con índices especializados, particionamiento mensual de la tabla orders, partición DEFAULT, migración de datos existentes y validación mediante consultas críticas con EXPLAIN (ANALYZE, BUFFERS). En MongoDB se definió un enfoque documental orientado a flexibilidad, consultas agregadas, disponibilidad mediante réplica set multi-AZ y estrategias diferenciadas de consistencia y lectura.

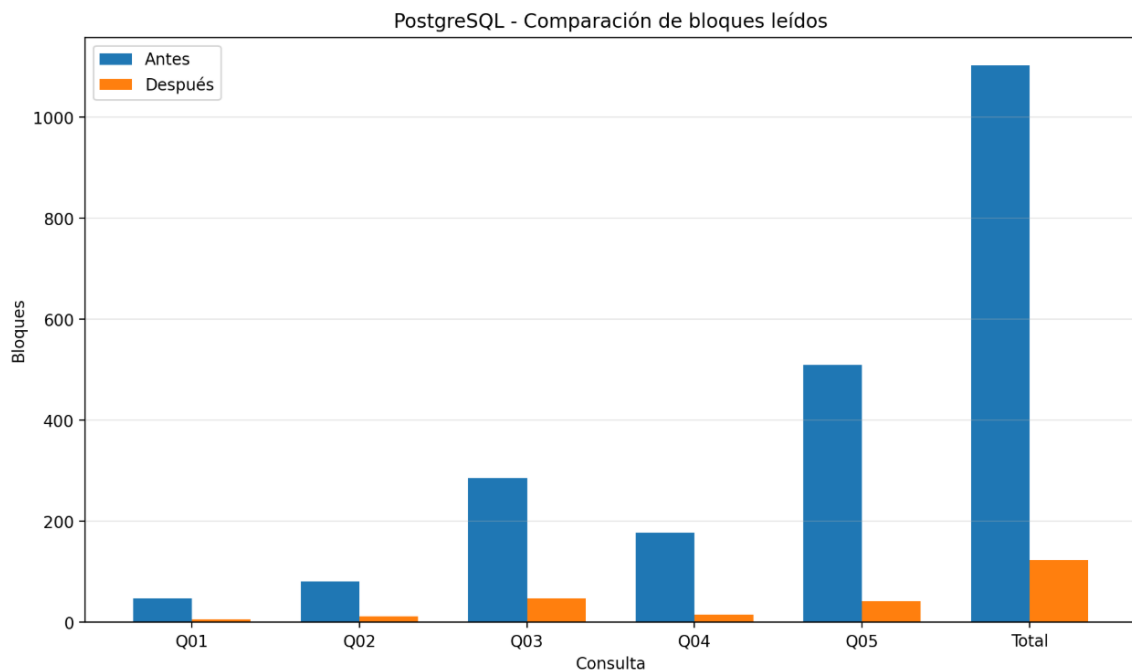
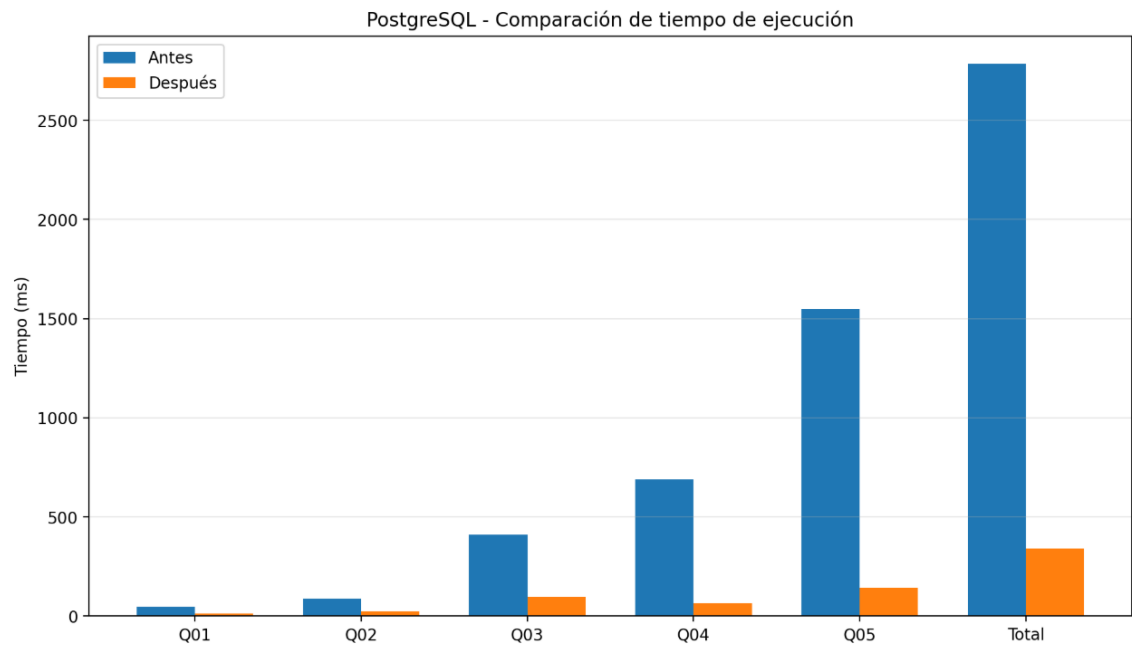
Como principal métrica de éxito, PostgreSQL evidenció una reducción del tiempo total de ejecución de 2.786,066 ms a 340,828 ms, equivalente a una mejora aproximada del 87,77%.

Adicionalmente, los bloques leídos disminuyeron de 1.103 a 123, logrando una reducción cercana al 88,85% en consumo de lectura. Estos resultados demuestran una mejora significativa en eficiencia, menor dependencia de escaneos secuenciales, mejor aprovechamiento de índices y mayor capacidad para soportar crecimiento histórico y consultas analíticas.

En términos de la arquitectura, el proyecto permitió validar una solución más escalable. PostgreSQL quedó orientado al manejo de integridad, consistencia transaccional y análisis relacional; mientras que MongoDB quedó posicionado para escenarios documentales, agregados, consultas de lectura flexible y alta disponibilidad.

Métrica / logro	Resultado destacado
Mejora en tiempo de ejecución PostgreSQL	87,77%
Tiempo antes de optimización	2.786,066 ms
Tiempo después de optimización	340,828 ms
Reducción global de bloques leídos	88,85%
Bloques antes	1.103
Bloques después	123
Tabla particionada principal	orders
Estrategia de particionamiento	RANGE mensual

Partición de control	orders_default
Motor transaccional	PostgreSQL / Supabase
Motor documental	MongoDB Atlas



4. Recomendaciones estratégicas clave.

Como recomendación principal, Ecommify debe mantener una arquitectura híbrida de datos donde PostgreSQL/Supabase continúe como motor transaccional principal para garantizar integridad, consistencia y trazabilidad del negocio, mientras que MongoDB Atlas se utilice para escenarios documentales, consultas agregadas, catálogos enriquecidos y modelos de lectura flexible y simples.

Esta división permite aprovechar las fortalezas de cada tecnología, evitando sobrecargar un único motor con responsabilidades incompatibles y habilitando una evolución más escalable del modelo de datos.

A nivel de rendimiento, se recomienda la estrategia de optimización aplicada: mantener índices selectivos, revisar periódicamente los planes de ejecución con EXPLAIN (ANALYZE, BUFFERS), monitorear el crecimiento de tablas e índices, automatizar la creación de particiones futuras y controlar la partición DEFAULT para evitar acumulación de registros fuera de rango. Adicionalmente, las consultas críticas deben ser evaluadas de forma recurrente.

5. Problema de negocio.

Ecommify enfrenta el reto de gestionar de forma eficiente muchos volúmenes de datos generados por una plataforma de comercio electrónico especializada en productos tecnológicos.

El negocio requiere procesar datos transaccionales críticos, como pedidos, pagos e inventario, con altos niveles de consistencia e integridad, al mismo tiempo que necesita explotar datos analíticos y semiestructurados, como catálogo extenso, reseñas y comportamiento de usuarios, para mejorar la toma de decisiones, la experiencia del cliente y la gestión comercial.

El problema está en que una única base de datos tradicional puede no ser suficiente para responder adecuadamente a estos diferentes patrones de uso, por lo que se requiere una arquitectura híbrida que permita equilibrar rendimiento, escalabilidad, flexibilidad y confiabilidad.

El negocio necesita una arquitectura de datos capaz de responder tanto a la operación diaria del ecommerce como al análisis avanzado de información para crecimiento comercial y mejora de experiencia de usuario.

6. Objetivos y alcance.

El objetivo principal es diseñar una arquitectura híbrida de bases de datos para Ecommify, que permita gestionar los datos transaccionales y analíticos de una plataforma de comercio electrónico, garantizando consistencia, escalabilidad, disponibilidad, rendimiento y flexibilidad en el manejo de información estructurada y documental, atributos de calidad importantes para la organización.

El alcance del proyecto está en el diseño de una solución de datos híbrida para Ecommify, incluyendo la creación del esquema relacional en PostgreSQL, la definición de colecciones documentales en MongoDB, la carga de datasets, la construcción de consultas críticas, la aplicación de índices, el diseño de particionamiento, la validación

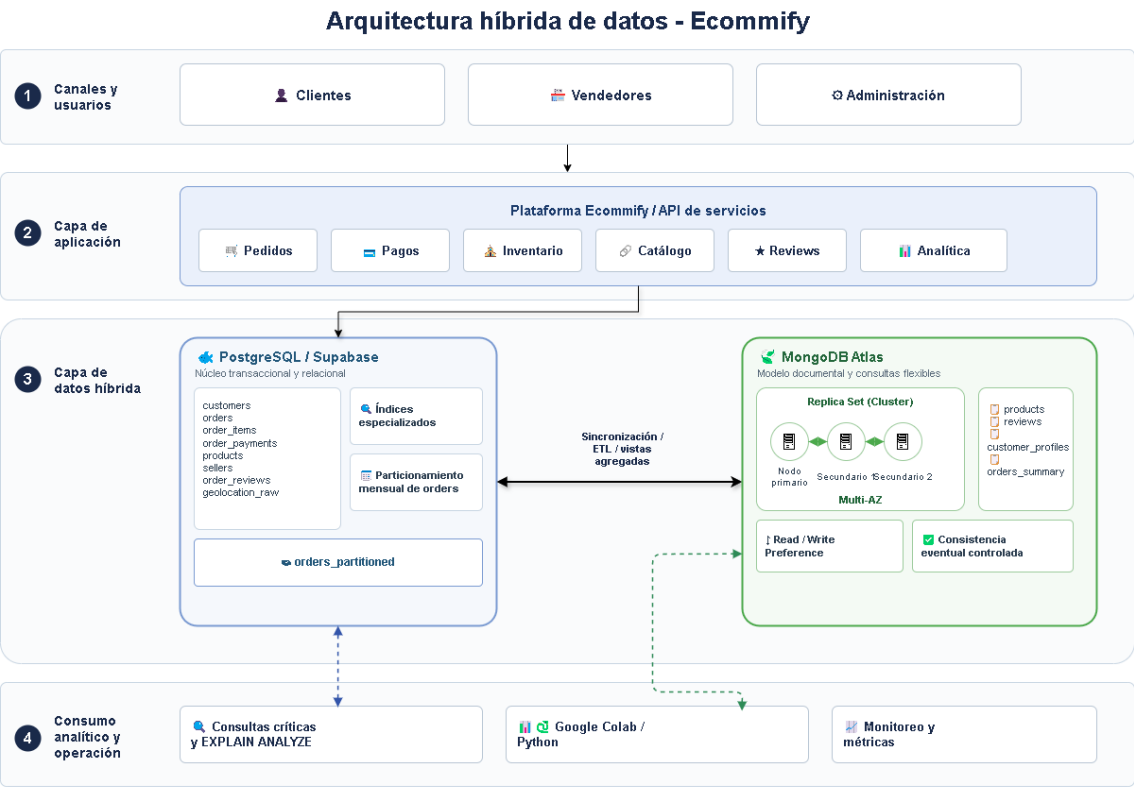
de rendimiento y la documentación de resultados. La solución cubre escenarios transaccionales como pedidos, pagos y productos, así como escenarios analíticos y documentales relacionados con catálogo, reseñas, comportamiento de usuarios y análisis histórico.

7. Metodología aplicada.

La metodología aplicada en el proyecto se desarrolló bajo un enfoque incremental, comparativo y orientado a evidencias, iniciando con el análisis del contexto de negocio y la clasificación de las cargas transaccionales, analíticas y documentales.

Con esto, se diseñó una arquitectura híbrida donde PostgreSQL/Supabase soporta el núcleo relacional y transaccional, mientras MongoDB Atlas atiende escenarios documentales y de consulta flexible. Se construyó el modelo de datos, se cargaron los datasets, se validaron conteos y se identificaron consultas críticas para medir rendimiento mediante EXPLAIN (ANALYZE, BUFFERS). Con base en los resultados, se aplicaron optimizaciones como índices especializados, reescritura de consultas y particionamiento mensual de orders. Finalmente, se consolidaron evidencias antes/después, gráficas, métricas de mejora y análisis de impacto, demostrando una solución eficiente, escalable.

8. Diagrama de arquitectura híbrida completa.



9. Matriz de decisiones: qué datos van a PostgreSQL vs MongoDB.

Dato	Motor recomendado	Justificación técnica	Uso principal
Clientes	PostgreSQL	Datos estructurados, relacionados con pedidos, pagos y análisis transaccional. Requieren integridad referencial.	Gestión de clientes, trazabilidad y relación con pedidos.
Pedidos	PostgreSQL	Entidad central del modelo transaccional. Requiere consistencia, control de estados, fechas, relaciones y particionamiento.	Operación comercial, logística, auditoría y análisis histórico.
Pagos	PostgreSQL	Información sensible y altamente transaccional. Requiere consistencia, precisión y relación directa con pedidos.	Conciliación, trazabilidad financiera y control transaccional.
Ítems de pedido	PostgreSQL	Relaciona pedidos, productos y vendedores. Requiere joins, agregaciones y consistencia relacional.	Análisis de ventas, detalle de compra y desempeño comercial.
Productos base	PostgreSQL	Datos maestros estructurados del catálogo, necesarios para integridad con ítems de pedido.	Administración del catálogo transaccional.
Vendedores	PostgreSQL	Datos estructurados relacionados con productos e ítems vendidos.	Gestión comercial y análisis de desempeño por vendedor.
Inventario	PostgreSQL	Requiere control transaccional, consistencia y actualización precisa.	Disponibilidad de productos y operación de ventas.
Geolocalización base	PostgreSQL	Alta volumetría, pero estructurada y útil para análisis por ubicación mediante consultas SQL.	Análisis regional, cobertura y segmentación geográfica.
Reseñas estructuradas	PostgreSQL	Cuando se requiere relación directa con pedidos y métricas como calificación promedio.	Análisis relacional de satisfacción y calidad.
Reseñas enriquecidas	MongoDB	Texto libre, comentarios, atributos variables y búsquedas documentales.	Análisis semiestructurado, sentimiento y consultas flexibles.

Catálogo extendido	MongoDB	Los productos tecnológicos pueden tener especificaciones variables por categoría. MongoDB permite documentos flexibles.	Catálogo enriquecido, filtros dinámicos y experiencia de usuario.
Perfil consolidado de cliente	MongoDB	Puede contener preferencias, historial resumido, comportamiento y atributos derivados.	Personalización, segmentación y lectura rápida.
Comportamiento de usuarios	MongoDB	Datos semiestructurados y de alto volumen, como eventos de navegación, búsquedas o interacciones.	Analítica de comportamiento y recomendaciones.
Documentos agregados de pedidos	MongoDB	Permite almacenar vistas consolidadas orientadas a lectura sin múltiples joins.	Consultas rápidas tipo “Order 360” o dashboards.
Métricas analíticas preprocesadas	MongoDB	Útil para almacenar agregados por período, categoría, cliente o región.	Dashboards, consultas rápidas y análisis exploratorio.
Logs o eventos de navegación	MongoDB	Estructura flexible, volumen alto y menor necesidad de integridad relacional estricta.	Observabilidad funcional y análisis de comportamiento.

10. Análisis del Teorema CAP aplicado por módulo

Módulo	Motor recomendado	Prioridad CA. P	Justificación
Pedidos	PostgreSQL	CP	Requiere consistencia fuerte en creación, estado, trazabilidad y relación con cliente, pagos e ítems.
Pagos	PostgreSQL	CP	Debe garantizar integridad financiera, evitar duplicidades y asegurar conciliación correcta.
Inventario	PostgreSQL	CP	La disponibilidad no debe comprometer la precisión del stock ni permitir sobreventa.
Clientes	PostgreSQL	CP	Requiere consistencia en la relación con pedidos, pagos y datos transaccionales.
Ítems de pedido	PostgreSQL	CP	Dependen directamente de pedidos, productos y vendedores; requieren integridad referencial.

Productos base	PostgreSQL	CP / CA operativa	Deben mantener consistencia para la operación comercial, aunque pueden replicarse hacia MongoDB para consulta.
Catálogo extendido	MongoDB	AP	Puede tolerar consistencia eventual; se prioriza disponibilidad y flexibilidad de atributos por categoría.
Reseñas enriquecidas	MongoDB	AP	No requieren consistencia inmediata; pueden replicarse y consultarse de forma eventual.
Comportamiento de usuarios	MongoDB	AP	Los eventos de navegación e interacción pueden capturarse con alta disponibilidad y procesarse posteriormente.
Perfiles consolidados	MongoDB	AP / CP según uso	Para personalización puede tolerar consistencia eventual; para datos sensibles debe consultarse la fuente relacional.
Analítica y reportes	PostgreSQL + MongoDB	AP para lectura / CP para fuente	Los reportes pueden usar datos agregados eventualmente consistentes, manteniendo PostgreSQL como fuente confiable.
Geolocalización	PostgreSQL / MongoDB	AP	Es información de referencia y consulta; puede priorizar disponibilidad y rendimiento de lectura.
Sincronización ETL	Capa intermedia	CP controlado	Debe preservar consistencia entre motores mediante validaciones, reintentos y control de errores.

CP : Consistencia y tolerancia a fallos de red

AP : Disponibilidad y tolerancia a fallos de red

11. Modelos ER y de documentos (sintéticos).

Diagrama ER en postgresql.

Diagrama Entidad-Relación (3FN)

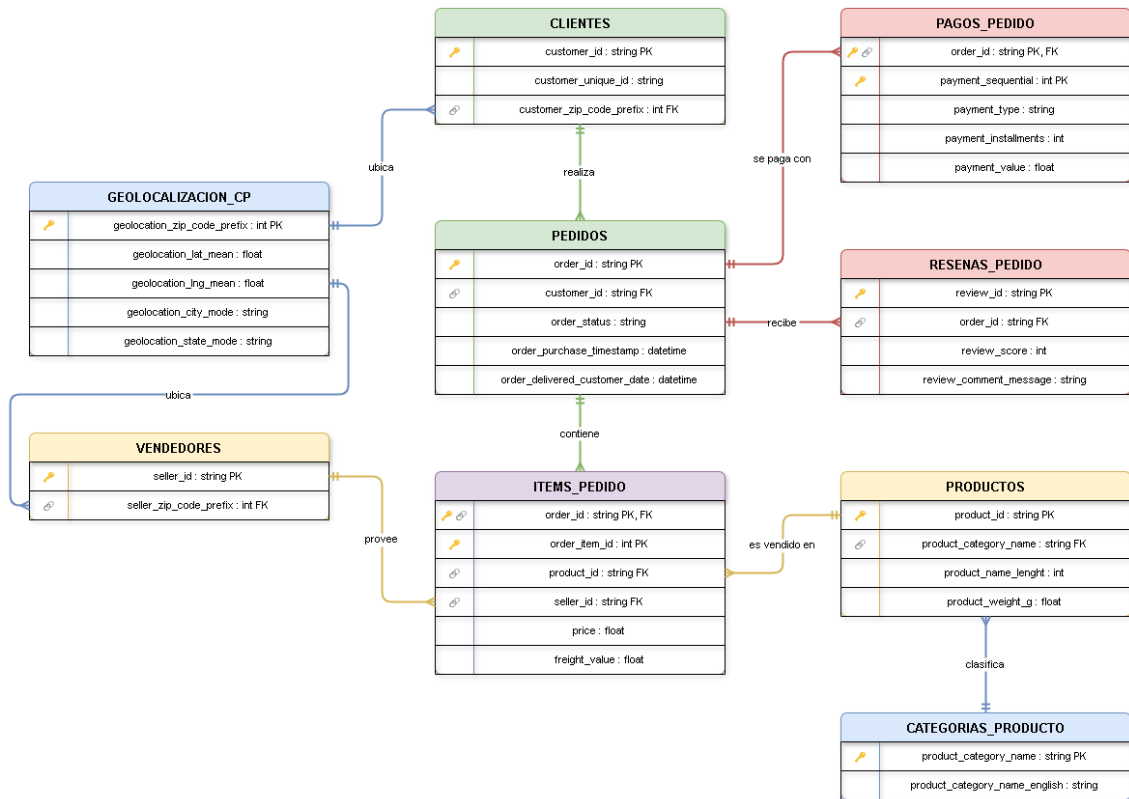
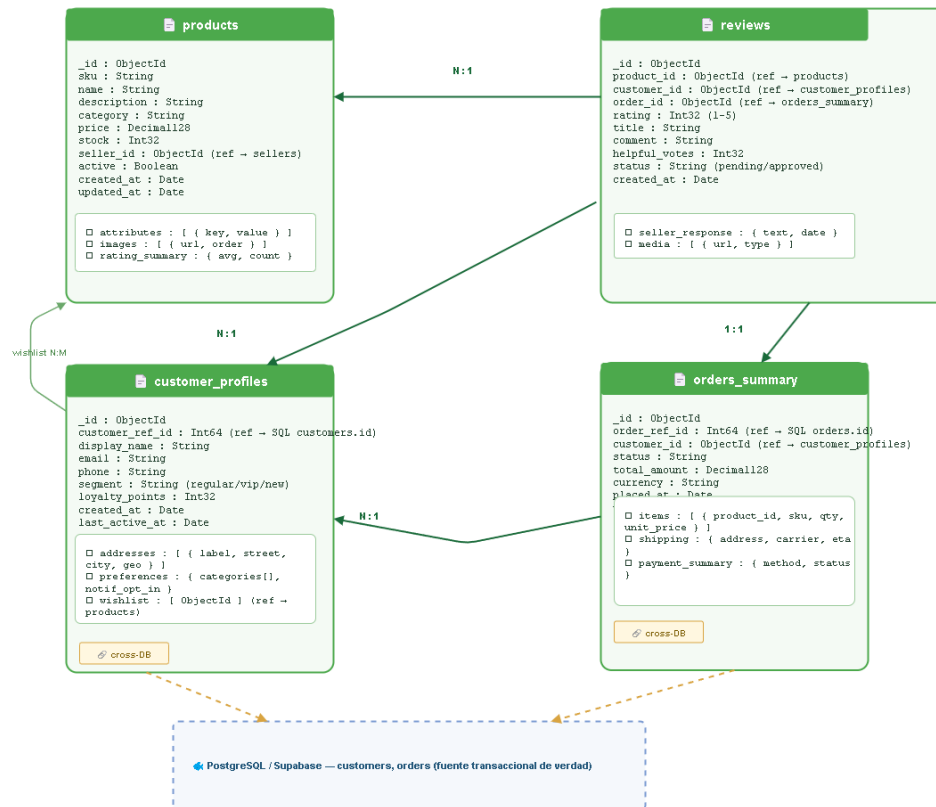


Diagrama ER en mongo.

MongoDB Atlas - Modelo de relaciones entre documentos (Ecommify)



12. Resumen de implementación PostgreSQL (tipos avanzados, extensiones, índices principales, particionamiento).

La implementación en PostgreSQL/Supabase para Ecommify se estructuró como el núcleo transaccional y relacional de la arquitectura híbrida, orientado a garantizar consistencia, integridad referencial, trazabilidad operativa y rendimiento en consultas críticas del negocio. Se modelaron las principales entidades del ecommerce —clientes, pedidos, productos, vendedores, pagos, ítems de pedido, reseñas y geolocalización— utilizando tipos de datos estructurados como TEXT, INTEGER, NUMERIC/DECIMAL, TIMESTAMP y campos derivados para análisis temporal y logístico. Adicionalmente, se consideró el uso de tipos avanzados como JSONB para atributos flexibles o datos semiestructurados, y TSVECTOR/búsqueda textual para mejorar el análisis de comentarios y reseñas.

A nivel de extensiones, se contemplaron capacidades propias de PostgreSQL para fortalecer la solución, como `pg_trgm` para búsquedas textuales más eficientes, `unaccent` para normalización de texto, `btree_gin` para soportar ciertos escenarios combinados de indexación y `pg_cron` como alternativa para automatizar tareas operativas, especialmente la creación futura de particiones. La estrategia de indexación se enfocó

en acelerar búsquedas por identificadores, filtros por estado, rangos de fecha, uniones entre tablas y análisis logístico, mediante índices B-tree simples, B-tree compuestos, índices parciales e índices GIN para búsquedas sobre texto o datos semiestructurados.

Como optimización estructural, se implementó particionamiento RANGE mensual sobre la tabla orders, usando order_purchase_timestamp como columna de partición. Esta decisión permitió organizar el histórico de pedidos por período, reducir el volumen de datos leído en consultas temporales, facilitar políticas de retención y archivado, y mejorar la eficiencia de operaciones analíticas y logísticas. También se definió una partición DEFAULT para capturar registros fuera de rango y evitar fallas en procesos de carga. En conjunto, la implementación PostgreSQL permitió mejorar el rendimiento, disminuir bloques leídos, optimizar planes de ejecución y preparar el modelo relacional para un crecimiento más controlado y escalable.

Componente	Implementación aplicada	Beneficio
Tipos avanzados	JSONB, TSVECTOR, TIMESTAMP, NUMERIC	Flexibilidad, análisis textual, precisión temporal y financiera
Extensiones	pg_trgm, unaccent, btree_gin, pg_cron	Búsqueda eficiente, normalización, soporte de índices y automatización
Índices principales	B-tree simples, compuestos, parciales y GIN	Menor tiempo de ejecución y reducción de escaneos secuenciales
Particionamiento	RANGE mensual sobre orders	Mejor rendimiento histórico, mantenimiento y escalabilidad
Partición de control	orders_default	Prevención de errores por registros fuera de rango

13. Resumen de implementación MongoDB (patrones de modelado, índices, aggregations).

La implementación de MongoDB Atlas para Ecommify se diseñó como la capa documental complementaria de la arquitectura híbrida, orientada a soportar información flexible, semiestructurada y agregada que no requiere el mismo nivel de normalización e integridad transaccional de PostgreSQL. El modelo documental se enfocó en colecciones como products, reviews, customer_profiles y orders_summary, permitiendo representar catálogos extendidos, reseñas enriquecidas, perfiles consolidados de clientes y vistas de pedidos orientadas a lectura rápida. Para el modelado se aplicaron patrones de embedding en datos que se consultan junto al documento principal, como atributos técnicos del producto, preferencias del cliente o ítems resumidos del pedido; y referencing cuando la relación puede crecer de forma significativa o requiere consulta independiente, como reseñas, productos, clientes y pedidos agregados.

A nivel de índices, se definieron índices sobre campos de alta frecuencia de consulta como product_id, category, customer_id, order_id, review_score, created_at y campos de análisis regional o temporal. Estos índices permiten reducir executionTimeMillis, disminuir documentos examinados y mejorar la eficiencia de búsquedas, filtros, ordenamientos y agregaciones. También se consideraron índices compuestos para consultas combinadas, por ejemplo categoría-fecha, cliente-fecha o estado-fecha, así

como índices de texto para búsquedas sobre comentarios, nombres de producto o descripciones. La estrategia de indexación se diseñó de forma controlada para evitar sobrecarga de escritura y consumo innecesario de almacenamiento, especialmente considerando las restricciones del free tier de MongoDB Atlas.

En cuanto a agregaciones, se implementaron pipelines con etapas como \$match, \$group, \$sort, \$project, \$lookup y \$limit para resolver análisis orientados a negocio, tales como productos más vendidos, reseñas promedio por producto, pedidos por estado, comportamiento de usuarios, métricas por categoría y desempeño de vendedores. Estas agregaciones permiten construir vistas documentales y reportes rápidos sin depender de múltiples uniones relacionales en tiempo de consulta. En conjunto, MongoDB aporta flexibilidad, disponibilidad y eficiencia para escenarios documentales y analíticos, complementando a PostgreSQL como fuente transaccional principal y habilitando una arquitectura más escalable, resiliente y alineada con diferentes patrones de acceso a datos.

Componente	Implementación aplicada	Beneficio
Modelo documental	Colecciones products, reviews, customer_profiles, orders_summary	Flexibilidad para datos semiestructurados y vistas orientadas a lectura
Patrón embedding	Atributos de producto, preferencias, ítems resumidos, datos derivados	Reduce consultas adicionales y mejora lecturas frecuentes
Patrón referencin	Relaciones entre productos, reseñas, clientes y pedidos agregados	Evita documentos excesivamente grandes y permite consultas independientes
Índices simples	product_id, customer_id, order_id, review_score, created_at	Acelera búsquedas, filtros y consultas puntuales
Índices compuestos	category + created_at, customer_id + order_date, status + date	Optimiza consultas combinadas y análisis por período
Índices de texto	Comentarios, nombres y descripciones	Habilita búsqueda flexible sobre contenido textual
Aggregations	\$match, \$group, \$sort, \$project, \$lookup, \$limit	Permite análisis de negocio, reportes y vistas documentales
Alta disponibilidad	Replica set multi-AZ	Mejora resiliencia y continuidad para consultas documentales

14. Metodología de pruebas aplicada.

La metodología de pruebas aplicada en Ecommify se estructuró bajo un enfoque comparativo, técnico y orientado a evidencias, con el propósito de validar la correcta implementación del modelo híbrido PostgreSQL/MongoDB y medir el impacto de las optimizaciones realizadas. Inicialmente se ejecutaron pruebas de carga y consistencia de datos, validando que los datasets fueran insertados correctamente en las tablas y colecciones definidas, mediante conteos, revisión de relaciones principales y verificación de registros críticos. Posteriormente, se ejecutaron consultas funcionales sobre los principales dominios del negocio, como clientes, pedidos, pagos, productos,

reseñas, vendedores y geolocalización, asegurando que el modelo respondiera adecuadamente a los patrones de consulta esperados.

En PostgreSQL/Supabase se aplicaron pruebas de rendimiento mediante EXPLAIN (ANALYZE, BUFFERS), comparando escenarios antes y después de la implementación de índices, reescritura de consultas y particionamiento mensual de la tabla orders. Las pruebas permitieron medir tiempos de ejecución, bloques leídos, tipos de operación utilizados por el optimizador, presencia de Seq Scan, uso de índices, eficiencia de JOIN, agregaciones y filtros por fecha. Esta comparación permitió cuantificar las mejoras obtenidas y validar que las optimizaciones redujeran el volumen de datos procesado y el costo de ejecución.

En MongoDB Atlas se realizaron pruebas orientadas a consultas documentales, agregaciones e índices, validando operaciones sobre colecciones como products, reviews, customer_profiles y orders_summary. Se revisaron métricas como tiempo de ejecución, documentos examinados, uso de índices y comportamiento de agregaciones con etapas \$match, \$group, \$sort, \$project y \$lookup. Adicionalmente, se consideraron criterios de disponibilidad, consistencia eventual, réplica set multi-AZ y preferencias de lectura/escritura, con el fin de evaluar el comportamiento esperado frente a consultas de lectura flexible y escenarios documentales.

Finalmente, los resultados fueron consolidados mediante tablas comparativas, gráficas de mejora, evidencias de ejecución en Supabase y análisis de impacto. Esta metodología permitió validar no solo que la solución funcionara correctamente, sino también que generara beneficios cuantificables en rendimiento, escalabilidad, eficiencia de lectura y separación adecuada de responsabilidades entre PostgreSQL y MongoDB.

Tipo de prueba	Motor	Objetivo	Evidencia generada
Carga de datos	PostgreSQL / MongoDB	Validar inserción correcta de datasets	Conteos por tabla/colección
Pruebas funcionales	PostgreSQL / MongoDB	Confirmar consultas de negocio	Resultados de consultas críticas
Rendimiento antes/después	PostgreSQL	Medir impacto de índices y particionamiento	EXPLAIN (ANALYZE, BUFFERS)
Uso de índices	PostgreSQL / MongoDB	Verificar reducción de escaneos y búsquedas eficientes	Planes de ejecución y métricas
Particionamiento	PostgreSQL	Validar distribución mensual y partición DEFAULT	Conteo por partición
Aggregations	MongoDB	Validar consultas documentales y analíticas	Pipelines \$match, \$group, \$sort
Disponibilidad y consistencia	MongoDB	Evaluar réplica set, read preference y consistencia eventual	Análisis CAP y configuración lógica

15. Resultados de pruebas de carga y escalabilidad.

Las pruebas de carga y escalabilidad realizadas para Ecommify permitieron validar el comportamiento de la arquitectura híbrida frente a volúmenes representativos de datos transaccionales, analíticos y documentales. En PostgreSQL/Supabase se cargaron datasets de alta volumetría, destacando geolocation_raw con aproximadamente 1.000.163 registros, order_items con 112.650 registros, order_payments con 103.886 registros y orders con 99.441 registros. La carga permitió confirmar que el modelo relacional soporta adecuadamente las entidades principales del negocio, siempre que se apliquen estrategias de optimización como índices, particionamiento, análisis de estadísticas y control de consultas críticas.

A nivel de rendimiento, PostgreSQL evidenció mejoras cuantificables después de aplicar índices especializados y particionamiento mensual sobre la tabla orders. El tiempo total de ejecución de las consultas críticas pasó de 2.786,066 ms a 340,828 ms, lo que representa una mejora aproximada del 87,77%. Adicionalmente, los bloques leídos disminuyeron de 1.103 a 123, con una reducción cercana al 88,85%. Estos resultados demuestran que la solución escala mejor al reducir el volumen de datos procesado, disminuir escaneos secuenciales y optimizar operaciones de filtrado, agregación, ordenamiento y unión entre tablas.

En MongoDB Atlas, las pruebas se hicieron en validar la capacidad del modelo documental para soportar consultas flexibles, agregaciones y lectura optimizada sobre colecciones como products, reviews, customer_profiles y orders_summary. La estrategia de índices permitió reducir la cantidad de documentos examinados y mejorar el tiempo de respuesta en consultas documentales y agregadas. Adicionalmente, el análisis de réplica set multi-AZ permitió validar una arquitectura orientada a disponibilidad, resiliencia y continuidad para cargas de lectura, considerando consistencia eventual, replication lag y preferencias de lectura/escritura según la criticidad de cada operación.

16. Análisis comparativo PostgreSQL vs MongoDB: tabla con aspectos evaluados, ganador por aspecto, justificación.

Aspecto evaluado	Ganador	Justificación
Consistencia transaccional	PostgreSQL	Es el motor más adecuado para operaciones críticas como pedidos, pagos e inventario, porque garantiza integridad referencial, transacciones ACID y control estricto del estado de los datos.
Integridad referencial	PostgreSQL	Permite definir llaves primarias, foráneas, restricciones y reglas de consistencia entre clientes, pedidos, productos, pagos y vendedores.
Consultas relacionales complejas	PostgreSQL	Resuelve mejor escenarios con múltiples JOIN, agregaciones SQL, filtros por fecha, análisis regional y reportes estructurados.
Rendimiento en consultas optimizadas	PostgreSQL	Con índices especializados y particionamiento mensual sobre orders, se evidenció una mejora

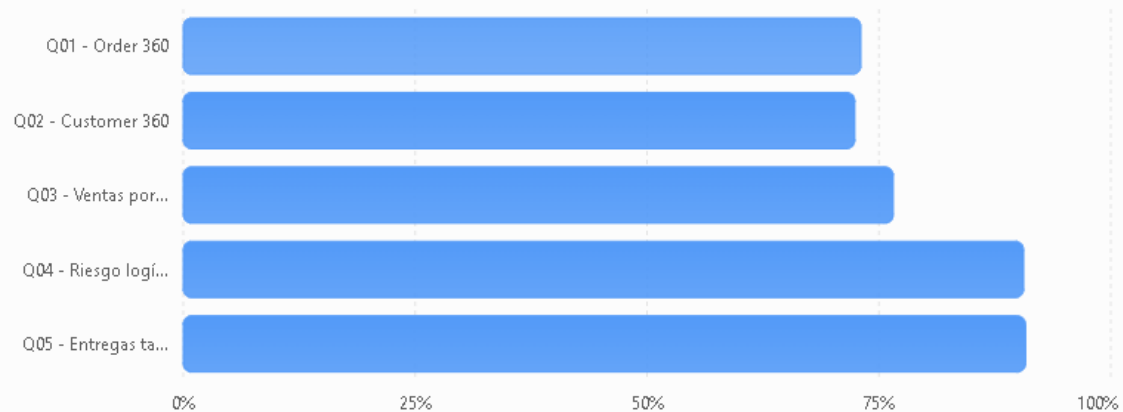
		global aproximada del 87,77% en tiempo de ejecución y 88,85% en bloques leídos.
Manejo de datos documentales y flexibles	MongoDB	Es más adecuado para catálogos extendidos, reseñas enriquecidas, perfiles consolidados y documentos con atributos variables.
Escalabilidad documental	MongoDB	Permite modelar documentos orientados al patrón de consulta, reduciendo la necesidad de múltiples uniones y facilitando lecturas rápidas sobre datos semiestructurados.
Disponibilidad y resiliencia distribuida	MongoDB	Su réplica set multi-AZ permite mejorar disponibilidad y continuidad operativa para cargas documentales y de lectura, considerando consistencia eventual.
Análisis histórico estructurado	PostgreSQL	El particionamiento por rango mensual facilita consultas por período, archivado, retención y administración del histórico transaccional.
Búsqueda textual y datos semiestructurados	MongoDB	Aunque PostgreSQL soporta GIN, JSONB y búsqueda textual, MongoDB resulta más natural para documentos, comentarios, atributos dinámicos y consultas flexibles.
Control financiero y conciliación	PostgreSQL	Los pagos requieren precisión, consistencia fuerte y trazabilidad, por lo que deben mantenerse en el motor relacional.
Catálogo enriquecido de productos	MongoDB	Los productos tecnológicos pueden tener especificaciones variables por categoría, lo cual se adapta mejor a documentos flexibles.
Comportamiento de usuarios	MongoDB	Eventos de navegación, preferencias, búsquedas e interacciones pueden almacenarse como documentos/eventos con estructura variable.
Monitoreo de rendimiento	PostgreSQL	EXPLAIN (ANALYZE, BUFFERS) permitió cuantificar con mayor precisión tiempos, bloques leídos y operaciones costosas. En MongoDB también se analizan métricas como executionTimeMillis, pero el free tier limita algunas capacidades avanzadas.
Manejo de consistencia eventual	MongoDB	Es más apropiado para módulos donde se tolera latencia de replicación, como reseñas, catálogo extendido o analítica no crítica.

17. Gráficas y métricas clave.

Categoría	Métrica	Resultado / lectura
PostgreSQL	Tiempo total antes	2.786,066 ms
PostgreSQL	Tiempo total después	340,828 ms
PostgreSQL	Mejora global en tiempo	87,77%
PostgreSQL	Bloques leídos antes	1.103 bloques
PostgreSQL	Bloques leídos después	123 bloques

PostgreSQL - Mejora por consulta crítica

Porcentaje de reducción del tiempo de ejecución después de aplicar índices y particionamiento.



La mejora global consolidada fue de 87,77%.

18. Trade-offs del Teorema CAP en cada módulo con análisis de escenarios de falla.

Módulo	Motor principal	Prioridad CAP	Escenario de falla	Trade-off principal	Estrategia de mitigación
Pedidos	PostgreSQL	CP	Falla de red o indisponibilidad parcial durante creación o actualización del pedido	Se sacrifica disponibilidad para evitar pedidos duplicados, incompletos o con estados inconsistentes	Transacciones ACID, validación de estado, reintentos controlados e idempotencia
Pagos	PostgreSQL	CP	Pérdida temporal de conexión con la base o con la pasarela de pago	Se prioriza consistencia financiera sobre disponibilidad inmediata	Control de duplicados, conciliación, auditoría e idempotencia de transacciones
Inventario	PostgreSQL	CP	Latencia o partición durante actualización de stock	Se evita vender productos sin disponibilidad real	Actualización atómica, restricciones, bloqueo controlado y validación previa

Clientes	PostgreSQL	CP	Falla durante actualización de datos maestros del cliente	Se prioriza integridad del dato sobre disponibilidad parcial	Llaves primarias, restricciones, validaciones y actualización centralizada
Ítems de pedido	PostgreSQL	CP	Falla durante asociación entre pedido, producto y vendedor	Se evita inconsistencia entre detalle de compra, pedido y pago	Integridad referencial, transacciones y validación de relaciones
Productos base	PostgreSQL	CP / CA operativa	Falla durante actualización de información maestra del producto	Se prioriza consistencia del catálogo operacional	Fuente maestra relacional y sincronización controlada hacia MongoDB
Catálogo extendido	MongoDB	AP	Latencia de replicación o falla entre nodos	Se mantiene disponibilidad aceptando datos eventualmente inconsistentes	Read preference controlado, índices por categoría y actualización asincrónica
Reseñas enriquecidas	MongoDB	AP	Retraso en replicación de nuevas reseñas	El usuario puede no ver la última reseña inmediatamente	Escritura en primary, lectura eventual, validación posterior e índices de texto
Comportamiento de usuarios	MongoDB	AP	Pérdida parcial de conectividad durante captura de eventos	Se prioriza disponibilidad de captura sobre consistencia inmediata	Ingesta asincrónica, colas, reintentos y procesamiento posterior
Perfiles consolidados	MongoDB	AP / CP según criticidad	Perfil desactualizado o por retraso de sincronización	Puede mostrarse una recomendación o preferencia no actualizada	Causal consistency en sesiones críticas y refresco programado
Analítica y reportes	PostgreSQL + MongoDB	AP para lectura / CP para fuente	Agregados no sincronizados en tiempo real	Los reportes pueden tener retraso, manteniendo la fuente transaccional confiable	ETL controlado, timestamps de actualización y reconciliación

					n contra PostgreSQL
Geolocalización	PostgreSQL / MongoDB	AP	Falla parcial en consulta de datos geográficos	Se acepta disponibilidad con posible retraso o dato no actualizado	Caché, replicación documental y validación periódica
Sincronización entre motores	Capa ETL / integración	CP controlado	Interrupción durante sincronización PostgreSQL → MongoDB	Se evita propagar datos incompletos o inconsistentes	Checkpoints, logs, reintentos, validaciones e idempotencia
Réplica set MongoDB multi-AZ	MongoDB Atlas	CP en escritura / AP controlado en lectura	Caída de nodo secundario o latencia entre zonas	Las lecturas pueden continuar; las escrituras críticas deben ir al primary	Write concern adecuado, lectura desde primary para operaciones sensibles y monitoreo de replication lag

CP : Consistencia y tolerancia a fallos de red

AP : Disponibilidad y tolerancia a fallos de red

19. Lecciones aprendidas sobre arquitecturas híbridas.

Lección aprendida	Aplicación en Ecommify	Impacto
No todos los datos requieren el mismo motor	PostgreSQL para transacciones; MongoDB para documentos y agregados	Mejor alineación entre tecnología y necesidad de negocio
La consistencia debe priorizarse por criticidad	Pedidos, pagos e inventario se mantienen en PostgreSQL	Menor riesgo operativo y financiero
La flexibilidad documental aporta valor en lectura	Catálogo extendido, reseñas y perfiles se modelan en MongoDB	Mayor adaptabilidad y menor dependencia de estructuras rígidas
El rendimiento depende del diseño físico	Índices, particionamiento y planes de ejecución en PostgreSQL	Reducción significativa de tiempos y bloques leídos
El modelado documental requiere criterio	Uso de embedding y referencing según crecimiento y consulta	Evita documentos excesivos y mejora consultas frecuentes

La sincronización es un punto crítico	Flujo controlado PostgreSQL → MongoDB	Evita inconsistencias entre motores
CAP debe analizarse por módulo	CP para transacciones; AP para lectura documental	Mejor balance entre consistencia y disponibilidad
El monitoreo debe ser transversal	EXPLAIN, métricas de índices, executionTimeMillis, replication lag	Mayor control operativo y capacidad de optimización
El free tier limita pruebas reales	Uso controlado de carga, índices y evidencias teóricas	Permite validar arquitectura sin sobredimensionar costos
La arquitectura híbrida requiere gobierno	Definir fuente de verdad, responsables y reglas de actualización	Reduce duplicidad, ambigüedad y riesgo de datos inconsistentes

20. Plan de escalamiento para crecimiento 10x.

Fase	Acción de escalamiento	Motor / componente	Objetivo	Resultado esperado
1	Medir volumetría actual y proyectada 10x	PostgreSQL / MongoDB	Identificar tablas y colecciones críticas	Priorización técnica por volumen e impacto
2	Mantener particionamiento mensual de orders	PostgreSQL	Reducir lectura de datos históricos	Mejor desempeño en consultas por fecha
3	Evaluar particionamiento de order_items, order_payments y geolocation_raw	PostgreSQL	Controlar crecimiento de tablas de alto volumen	Menor costo de consulta y mantenimiento
4	Automatizar creación de particiones futuras	PostgreSQL / pg_cron	Evitar intervención manual	Operación más estable y preventiva
5	Optimizar índices simples, compuestos y parciales	PostgreSQL	Acelerar filtros, JOIN y consultas críticas	Menos Seq Scan y menor lectura de bloques
6	Revisar planes con EXPLAIN (ANALYZE, BUFFERS)	PostgreSQL	Detectar operaciones costosas	Ajuste continuo de consultas e índices
7	Crear estrategia de archivado histórico	PostgreSQL	Separar datos operativos de históricos	Menor carga sobre tablas activas
8	Separar cargas analíticas de cargas transaccionales	PostgreSQL / MongoDB	Evitar impacto sobre operaciones críticas	Mayor estabilidad del core transaccional
9	Optimizar colecciones documentales	MongoDB	Ajustar embedding y referencing	Documentos más eficientes y mantenibles
10	Crear índices selectivos en campos de consulta frecuente	MongoDB	Reducir documentos examinados	Menor executionTimeMillis

11	Optimizar pipelines de agregación	MongoDB	Mejorar reportes y vistas documentales	Aggregations más rápidas y controladas
12	Monitorear replication lag y read preferences	MongoDB Atlas	Controlar consistencia eventual	Lecturas más confiables según criticidad
13	Escalar tier de base de datos	Supabase / MongoDB Atlas	Superar límites de free tier	Mayor CPU, memoria, almacenamiento y monitoreo
14	Evaluar sharding en colecciones de alto crecimiento	MongoDB	Distribuir carga horizontalmente	Mayor capacidad para lecturas y escrituras masivas
15	Implementar monitoreo centralizado	PostgreSQL / MongoDB	Controlar rendimiento, errores y saturación	Gestión proactiva del crecimiento

Escenario 10x	Riesgo principal	Mitigación recomendada
Aumento de pedidos	Consultas lentas sobre orders	Particionamiento, índices por fecha/estado y archivado
Aumento de pagos	Riesgo de duplicidad o conciliación lenta	Idempotencia, consistencia fuerte e índices por order_id
Aumento de ítems de pedido	Mayor costo de JOIN	Índices compuestos y posible particionamiento
Aumento de reseñas	Consultas textuales lentas	MongoDB con índices de texto y agregaciones optimizadas
Aumento de catálogo	Atributos variables y documentos grandes	Modelo documental controlado y referencing selectivo
Aumento de consultas analíticas	Sobrecarga del core transaccional	Vistas agregadas, MongoDB y procesos ETL
Aumento de geolocalización	Alto volumen de lectura	Agregación, deduplicación, índices y posible particionamiento
Aumento de usuarios concurrentes	Saturación de conexiones	Pooling, escalamiento de tier y separación de cargas
Crecimiento de almacenamiento	Costos y mantenimiento elevados	Retención, archivado y compresión lógica
Mayor distribución geográfica	Latencia y consistencia eventual	Réplica set multi-AZ, read preferences y monitoreo de lag

21. Migración de free tier a producción: consideraciones técnicas y de costo.

Dimensión	Free tier	Producción recomendada	Impacto técnico / costo
Cómputo	Capacidad limitada	Instancias con CPU y memoria dedicadas	Mayor rendimiento, pero mayor costo mensual
Almacenamiento	Límite reducido	Capacidad escalable según crecimiento	Costo aumenta por volumen y retención

Concurrencia	Restricciones de conexiones y carga	Pooling y mayor capacidad de conexiones	Soporta más usuarios y procesos simultáneos
Alta disponibilidad	Limitada o básica	Multi-AZ / réplica set dedicado	Mejora continuidad, incrementa costo
Backups	Básicos o limitados	Backups automáticos y retención definida	Reduce riesgo operativo, aumenta almacenamiento
Monitoreo	Métricas limitadas	Alertas, métricas avanzadas y trazabilidad	Mayor control operativo y detección temprana
Seguridad	Configuración básica	Roles, secretos, cifrado, redes privadas y auditoría	Necesario para ambiente productivo
Índices	Validados en pruebas	Ajustados según carga real	Mejora rendimiento, consume almacenamiento
Particionamiento	Implementado de forma controlada	Automatizado y monitoreado	Reduce degradación por crecimiento
MongoDB Atlas	Free/shared cluster	Clúster dedicado con réplica set multi-AZ	Mayor disponibilidad y control de rendimiento
PostgreSQL/Supabase	Validación académica/técnica	Plan productivo con backups y escalamiento	Base confiable para transacciones críticas
Ambientes	Uno solo o limitado	Dev, QA/Staging y Producción	Mayor gobierno, mayor costo operativo
Transferencia de datos	Baja volumetría	Tráfico entre aplicaciones, BI, ETL y usuarios	Puede generar costos adicionales
Soporte	Limitado	Soporte operativo y SLA formal	Relevante para continuidad del negocio
Escalamiento	Manual o limitado	Escalamiento vertical/horizontal controlado	Permite crecimiento 10x con menor riesgo

22. Tecnologías complementarias recomendadas (caching, search, observabilidad).

Categoría	Tecnología recomendada	Aplicación en Ecommify	Beneficio esperado	Consideraciones técnicas
Caching	Redis / Valkey	Caché de catálogo, productos más consultados, sesiones y consultas frecuentes	Reduce latencia y disminuye carga sobre PostgreSQL y MongoDB	Definir TTL, invalidación, patrón cache-aside y control de datos sensibles

Caching	CDN / Edge Cache	Imágenes de productos, contenido estático y recursos públicos	Mejora tiempos de carga para usuarios finales	No usar para pagos, pedidos o datos personalizados sensibles
Search	OpenSearch / Elasticsearch	Búsqueda avanzada de productos, reseñas, categorías y texto libre	Mejora experiencia de búsqueda y evita sobrecargar las bases principales	Requiere sincronización, reindexación y control de consistencia
Search	MongoDB Atlas Search	Búsqueda textual sobre productos, reseñas y perfiles documentales	Facilita consultas flexibles sobre documentos MongoDB	Recomendado cuando la búsqueda se concentra en colecciones MongoDB
Observabilidad	OpenTelemetry	Instrumentación de APIs, ETL y sincronización PostgreSQL → MongoDB	Permite trazabilidad distribuida extremo a extremo	Estandarizar correlation-id, métricas, logs y trazas
Observabilidad	Prometheus + Grafana	Dashboards, métricas técnicas y alertas operativas	Facilita monitoreo proactivo del sistema	Definir métricas de latencia, errores, CPU, memoria, conexiones y saturación
Observabilidad	Datadog / New Relic	APM, logs, infraestructura y monitoreo de experiencia de usuario	Mayor visibilidad en ambientes productivos con SLA	Puede incrementar costos; requiere gobierno FinOps
Logs	Loki / ELK / OpenSearch Dashboards	Centralización de logs de aplicación, base de datos, ETL y errores	Mejora diagnóstico, auditoría y análisis de fallas	Normalizar estructura de logs y definir retención
Mensajería / eventos	Kafka / RabbitMQ / Azure Service Bus	Sincronización asincrónica entre PostgreSQL, MongoDB, search y analítica	Desacopla cargas y mejora resiliencia	Implementar idempotencia, reintentos, DLQ y monitoreo
Seguridad	Vault / Azure Key Vault / Secret Manager	Gestión de credenciales, cadenas de conexión y secretos	Reduce riesgo de exposición de credenciales	Integrar con CI/CD y rotación de secretos
BI / Analítica	Power BI	Reportes ejecutivos sobre	Facilita explotación	Preferir vistas, agregados o

		datos agregados	de datos para negocio	réplicas para no afectar el core transaccional
--	--	-----------------	-----------------------	--

23. Estrategia de CI/CD para cambios de esquema.

La estrategia de CI/CD para cambios de esquema en Ecommify debe garantizar que las modificaciones en PostgreSQL y MongoDB se ejecuten de forma controlada. El objetivo principal es evitar cambios manuales directamente sobre las bases de datos, reducir errores en producción y asegurar que cada ajuste de tablas, colecciones, índices, particiones o agregaciones pase por validaciones técnicas antes de ser desplegado.

Para PostgreSQL, los cambios de esquema deben gestionarse mediante scripts versionados de migración, incluyendo creación o modificación de tablas, índices, restricciones, particiones, funciones y vistas. Cada cambio debe almacenarse en el repositorio Git.

Para MongoDB, la estrategia debe enfocarse en versionar cambios de colecciones, validadores de esquema, índices, pipelines de agregación y scripts de transformación documental. Debido a que MongoDB permite estructuras flexibles, es fundamental definir reglas mínimas de validación, mantener compatibilidad con versiones anteriores de documentos y aplicar migraciones progresivas para evitar afectar consultas existentes. Los índices deben probarse previamente para validar impacto en lectura, escritura y almacenamiento.

24. Cumplimiento de objetivos.

Objetivo	Cumplimiento	Evidencia / resultado
Diseñar una arquitectura híbrida PostgreSQL/MongoDB	Cumplido	Separación entre núcleo transaccional relacional y capa documental flexible
Implementar PostgreSQL como base transaccional	Cumplido	Modelo relacional con integridad, consistencia, índices y particionamiento
Implementar MongoDB como capa documental	Cumplido	Colecciones para catálogo extendido, reseñas, perfiles y agregados
Optimizar el rendimiento de consultas críticas	Cumplido	Mejora aproximada de 87,77% en tiempo y 88,85% en bloques leídos
Evaluar escalabilidad, CAP y migración a producción	Cumplido	Análisis por módulo, plan 10x, trade-offs CAP y consideraciones técnicas/costo

25. Principales aprendizajes del equipo.

Aprendí que una arquitectura híbrida de datos permite aprovechar de forma más efectiva las fortalezas de cada motor, utilizando PostgreSQL para procesos transaccionales que requieren consistencia e integridad, y MongoDB para escenarios documentales, flexibles y orientados a lectura.

También se evidenció que el rendimiento no depende únicamente de la tecnología seleccionada, sino del diseño aplicado: índices adecuados, particionamiento, modelado

documental, agregaciones optimizadas y monitoreo continuo son factores determinantes para lograr escalabilidad.

Finalmente, se aprendió que las decisiones de arquitectura deben evaluarse desde una perspectiva 360, considerando consistencia, disponibilidad, costos, operación, crecimiento futuro y escenarios de falla.

26. Reflexión final sobre el proceso.

El desarrollo del proyecto Ecommify permitió consolidar una visión integral sobre el diseño de bases de datos modernas, evidenciando que la selección tecnológica debe responder a las necesidades del negocio, los patrones de consulta, la criticidad de los datos y los escenarios de crecimiento. La implementación híbrida entre PostgreSQL y MongoDB demostró que no existe un único motor ideal para todos los casos, sino que cada tecnología aporta valor cuando se utiliza en el contexto adecuado.

Durante el proceso se fortaleció la capacidad de análisis arquitectónico, especialmente en aspectos como modelado relacional, modelado documental, optimización de consultas, particionamiento, índices, consistencia, disponibilidad y escalabilidad. Las pruebas realizadas permitieron validar con evidencias que las decisiones de diseño impactan directamente el rendimiento, la mantenibilidad y la preparación de la solución para escenarios productivos.

Referencias:

1. PostgreSQL Global Development Group. (2026). PostgreSQL documentation. PostgreSQL. ([PostgreSQL][1])
2. Supabase. (2026). Database migrations. Supabase Docs. ([Supabase][2])
3. MongoDB, Inc. (2026). MongoDB documentation. MongoDB Docs. ([MongoDB][3])
4. MongoDB, Inc. (2026). Indexes. MongoDB Manual. ([MongoDB][4])
5. Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. Computer, 45(2), 23–29. ([UCSB Computer Science][5])
6. Redis. (2026). Fast caching solutions with Redis. Redis Docs. ([Redis][6])
7. OpenTelemetry. (2026). OpenTelemetry documentation. OpenTelemetry Docs. ([opentelemetry.io][7])